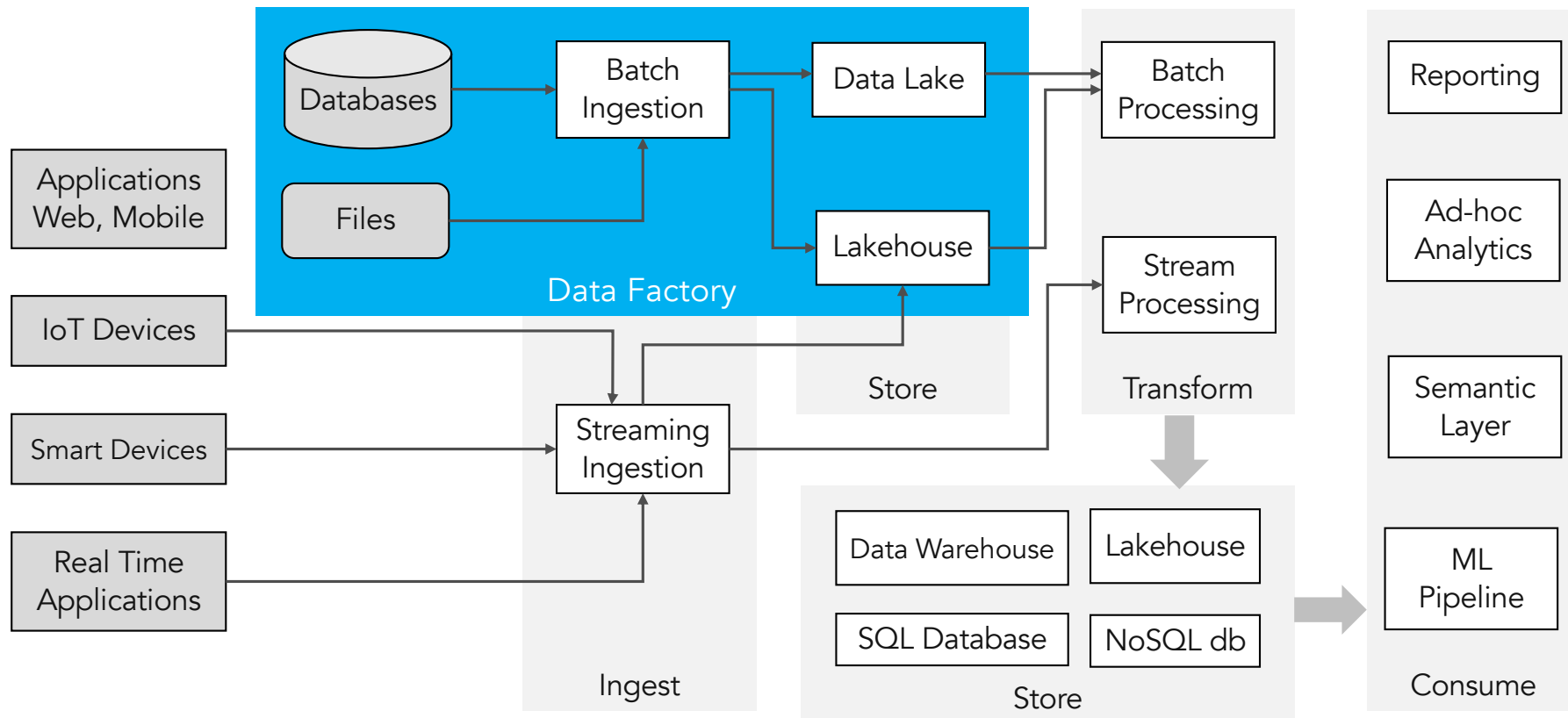


Fabric Data Factory

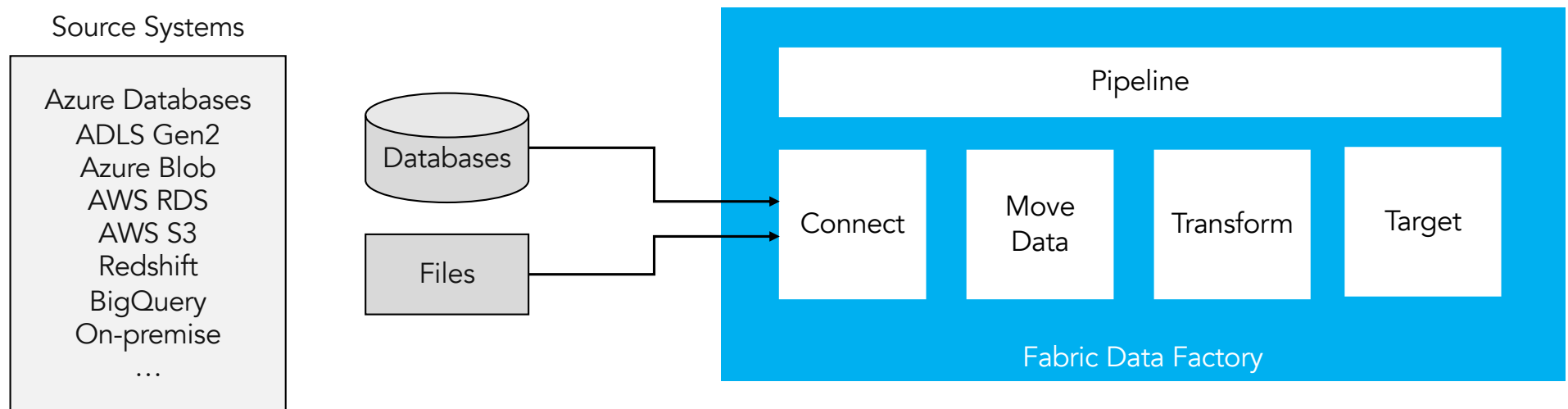
www.3aayaam.in

Data Factory



Data Factory

- Data Integration Solution.
- Data Integration – Bring data from all places together for further analysis and processing.
- Batch ingestion.

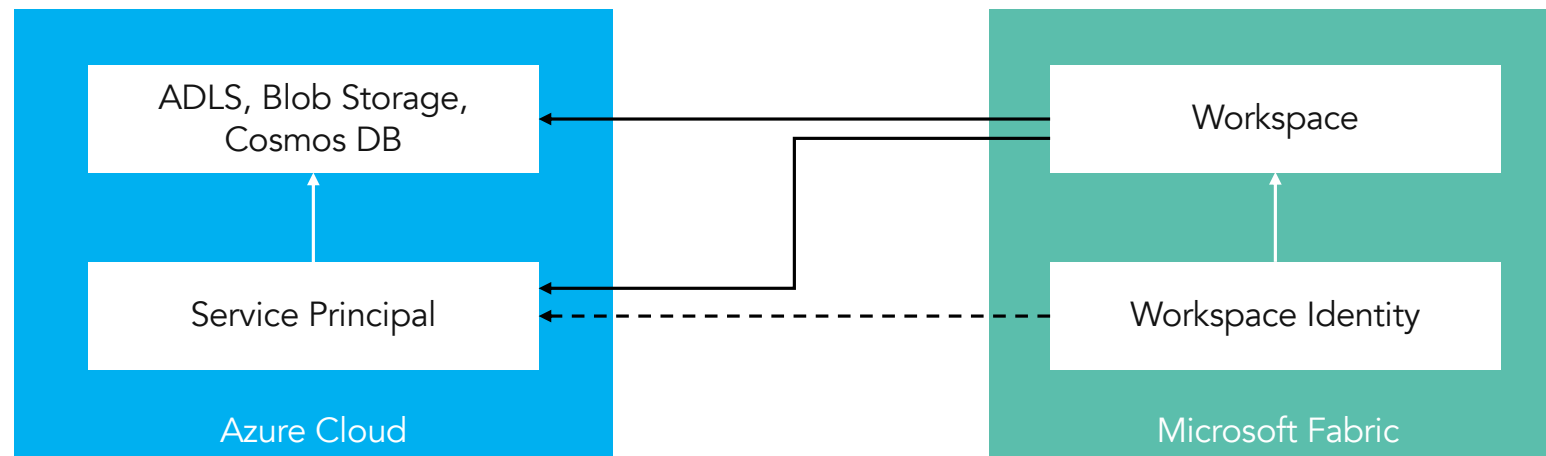


Data Sources

- Connectors.
- Supported sources:
 - Azure – SQL DB, Blob storage, ADLS, Cosmos DB, Data Explorer, Tables, Databricks, KQL database etc.
 - AWS RDS, Redshift, Teradata, BigQuery, PostgreSQL, Oracle, ClickHouse. MongoDB etc.
 - Salesforce, Asana, Zoho, Hive, Google Analytics,
- Authentication :
 - Service Principal – Azure Service Principal.
 - Workspace Identity – Service principal for workspace.
 - Organizational Account – Entra ID (Azure AD) user and pwd.

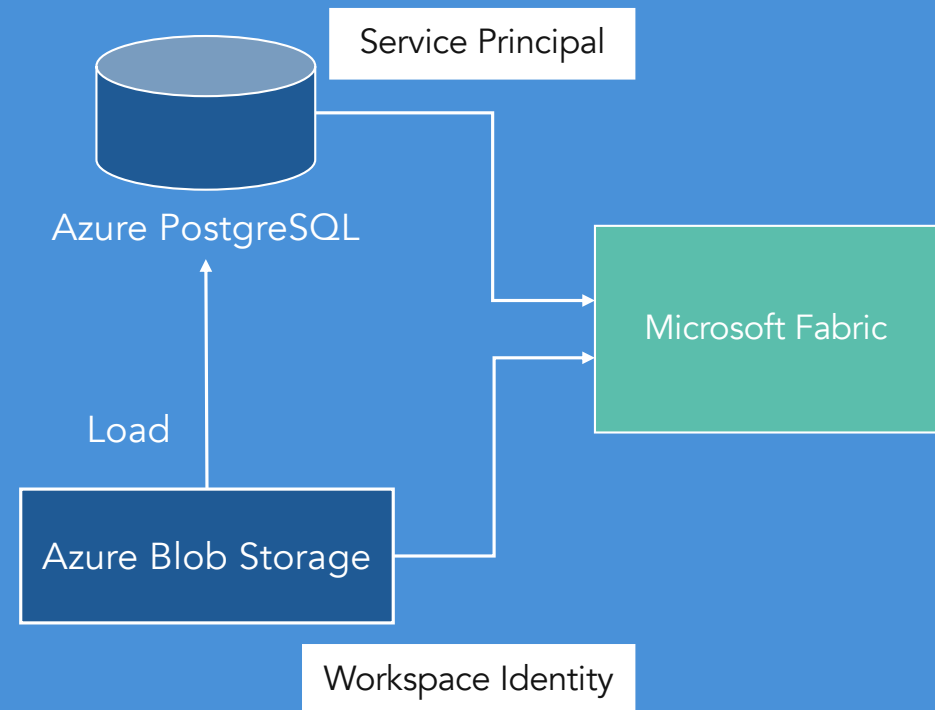
Connect to Data Sources

- Service Principal – Azure identity.
- Workspace Identity – Managed service principal for Fabric workspace.
- AWS, GCP, SAP, On-premise etc.



HANDS-ON : Create Data Sources

- Azure PostgreSQL database.
 - Create Azure PostgreSQL resource.
 - Modify Server Parameters – `shared_preload_libraries`, `azure.extensions`.
 - Restart server.
 - Follow command sheet.
- Azure Blob storage.
- Service Principal for PostgreSQL.
- Workspace Identity for Blob Storage.



EXERCISE : Create Data Sources

- Load the remaining files to "account" and "customer" tables.
- Create a new Azure PostgreSQL Server.
- Create database transactiondb;
- Create "ledger" table. Load all 5 files.
- Create "transaction" table. Load all 5 files.
- Loan data should be in Storage Account. Upload the 2nd file also.

Fabric Copy Job

www.3aayaam.in

Move Data – Copy Job



- Modes – Full copy & Incremental copy. No Pipeline is needed.
- Incremental CDC copy. CDC must be enabled at source. column (datetime) need to be selected for tracking data changes.
- Watermark based copy. Column (datetime) need to be selected for tracking data changes.
- Target write method – Append, Merge, overwrite.
- Supported sources – Azure SQL DB, SQL Server, Azure SQL MI, Lakehouse Table, Snowflake, BigQuery.
- CI/CD integration with Git and Azure DevOps.

Move Data – Copy Job Activity

- Full copy & Incremental copy (Copy Job) using Pipeline.

Enable CDC at source SQL Server database.

```
USE financedb
GO
EXEC sys.sp_cdc_enable_db
GO
```

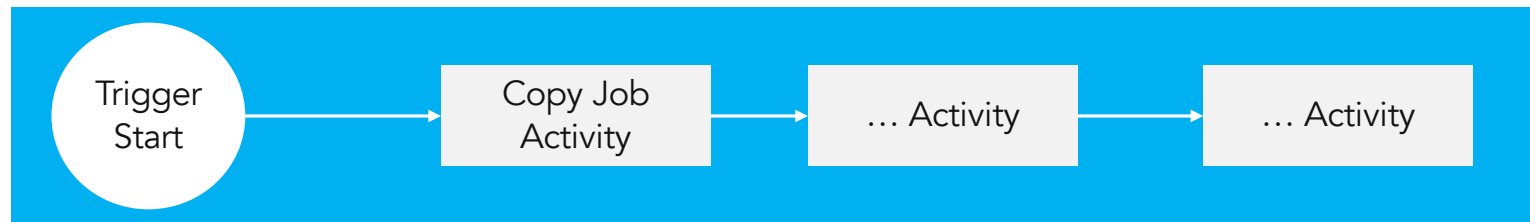
HANDS-ON : Copy Job

- Copy records from PostgreSQL "account" & "customer" table to a Lakehouse.
- Copy files from Blob storage to Lakehouse.

Fabric Copy Activity

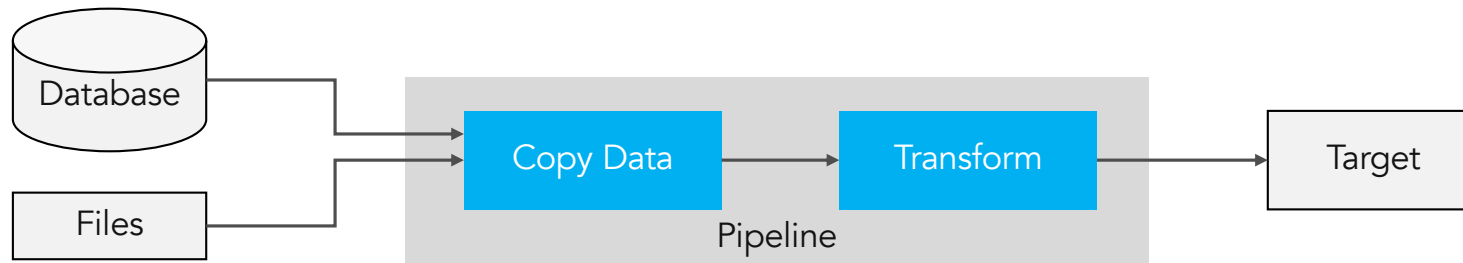
www.3aayaam.in

Pipeline



- Start or Trigger Pipeline.
- Executes all activities one after another or based on condition.
- Copy Job, Copy Activity, Dataflow (Gen2) etc.
- Conditional execution – based on success, failure or other conditions.

Move Data – Copy Activity



- Copy data, perform Transformation and move to destination using Pipeline.
- CDC or Incremental copy is not supported.
- Dynamic content, parameters, variables, functions can be passed.
- Parallel copy – Degree of copy parallelism.
- Created using "Pipeline".
- Pipelines handle large amount of data, performant, cost-effective. Scalable ingestion pipeline.

Performance

- Intelligent Throughput Optimization – Optimizes throughput by modifying CPU, Memory and Network.

Value	Description
Auto	Allow the service to dynamically apply the optimal throughput optimization based on your <u>source-destination pair and data pattern</u> .
Standard	Allow the service to dynamically apply the throughput optimization under standard compute resources based on your <u>source-destination pair and data pattern</u> .
Balanced	Allow the service to dynamically apply throughput optimization which balances the throughput and available compute resources based on your <u>source-destination pair and data pattern</u> .
Maximum	Allow the service to dynamically apply the throughput optimization by utilizing the maximum available compute resources based on your <u>source-destination pair and data pattern</u> .

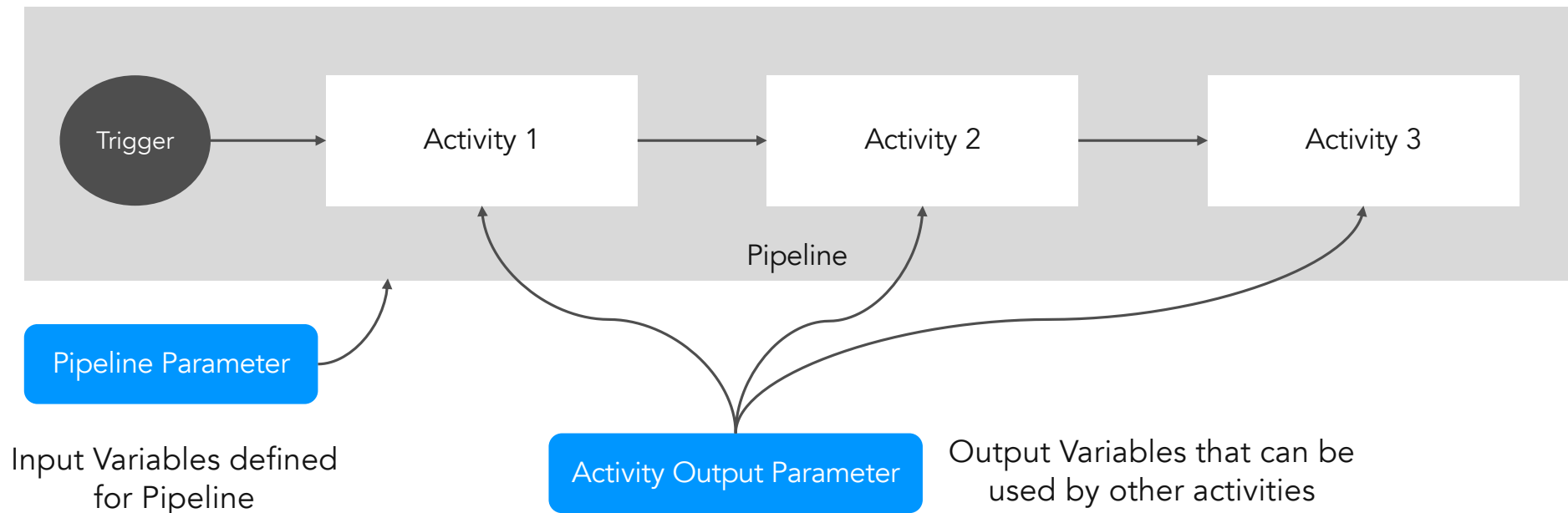
Performance

- Parallel Copy – No. of parallel threads to be used to read from sources and write to destinations.
- By default, parallelism is automatically applied based on source-destination pair and data pattern.



Parameters

- Used in Pipeline and Activity.
- Values can be passed dynamically. Templatize pipeline.



System Parameters

Variable Name	Description
@pipeline().DataFactory	Name of the data or Synapse workspace the pipeline run is running in
@pipeline().Pipeline	Name of the pipeline
@pipeline().RunId	ID of the specific pipeline run
@pipeline().TriggerId	ID of the trigger that invoked the pipeline
@pipeline().TriggerName	Name of the trigger that invoked the pipeline
@pipeline().TriggerTime	Time of the trigger run that invoked the pipeline. This is the time at which the trigger actually fired to invoke the pipeline run, and it may differ slightly from the trigger's scheduled time.
@pipeline().GroupId	ID of the group to which pipeline run belongs. In Microsoft Fabric, a 'group' refers to a collection of related resources that can be managed together. Groups are used to organize and control access to resources, making it easier to manage permissions and monitor activities across multiple pipelines.
@pipeline()?.TriggeredByPipelineName	Name of the pipeline that triggers the pipeline run. Applicable when the pipeline run is triggered by an ExecutePipeline activity. Evaluate to <i>Null</i> when used in other circumstances. Note the question mark after @pipeline()
@pipeline()?.TriggeredByPipelineRunId	Run ID of the pipeline that triggers the pipeline run. Applicable when the pipeline run is triggered by an ExecutePipeline activity. Evaluate to <i>Null</i> when used in other circumstances. Note the question mark after @pipeline()

Expressions

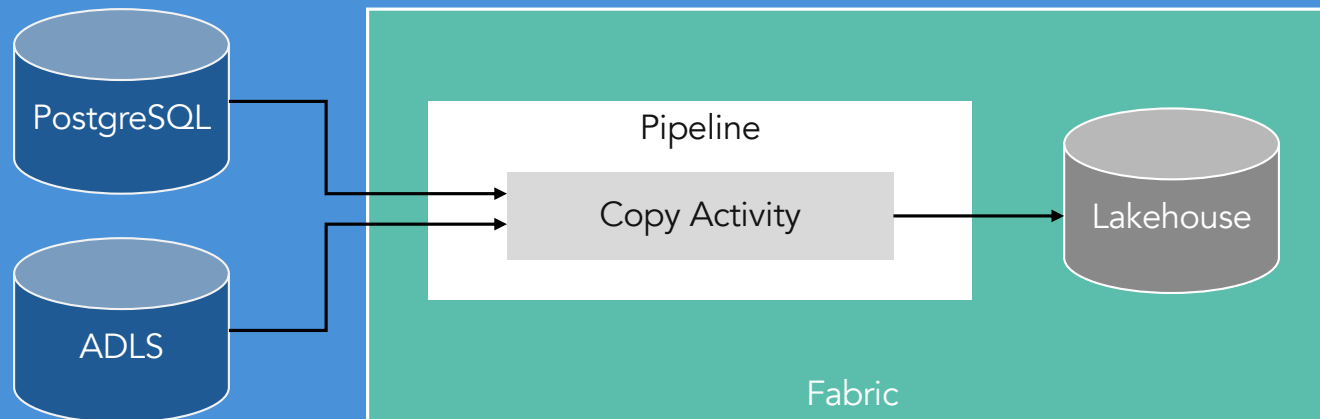
- References input variables/parameters.
- Functions, logic used to calculate input variables and other values at run time.
- Reference Pipeline parameter :
 - `@pipeline().parameters.<variable-name>`
- Reference Pipeline Variable
 - `@variables('<var-name>')`
- Reference previous Activity output :
 - `@activity('<previous-activity-name>').output.<value-name>`
- Functions available for manipulations : Date functions, String functions, Math functions, Conversion functions, Array & Dictionary functions, Compare functions.

Copy Job or Copy Activity

If you only need to copy your data and don't need transformations, a **Copy job** might be a better option for you. Copy jobs provide a simplified experience for data movement scenarios that don't require creating a full pipeline.

HANDS-ON : Copy Activity

- Copy from PostgreSQL & ADLS to Lakehouse.
- Use parameters to templatize the pipeline.
- Use Pipeline for incremental copy from ADLS to Lakehouse. (After pipeline topic)
- Use Pipeline for incremental copy from PostgreSQL to Lakehouse. (After pipeline topic)



EXERCISE : Copy Activity

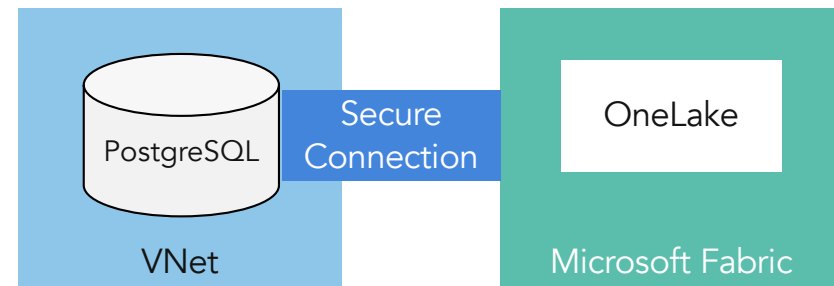
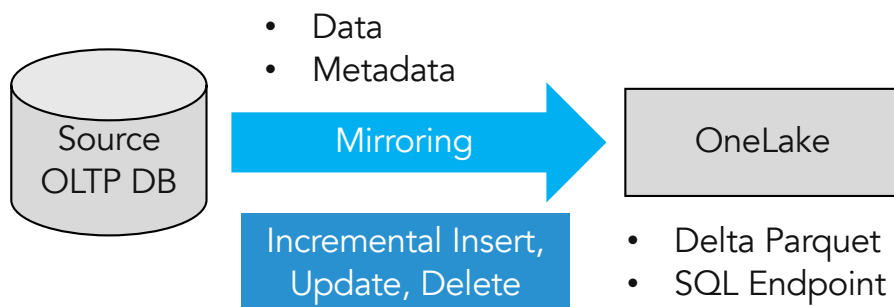
- Copy from PostgreSQL account table and the ledger ADLS files to Lakehouse.
- Use parameters to copy PostgreSQL customer table and ADLS transaction files.

Fabric Mirroring

www.3aayaam.in

Move Data – Mirroring

- Continuously replicate data from source databases.
- Move data changes (Insert, Update, Delete) in near real-time.
- Data is landed on OneLake in Delta Parquet table format. Format conversion is handled by mirroring.
- Provides SQL Analytics Endpoint. T-SQL or visual query editor can be used to query data from mirrored database.
- Entire database on individual tables can be mirrored.

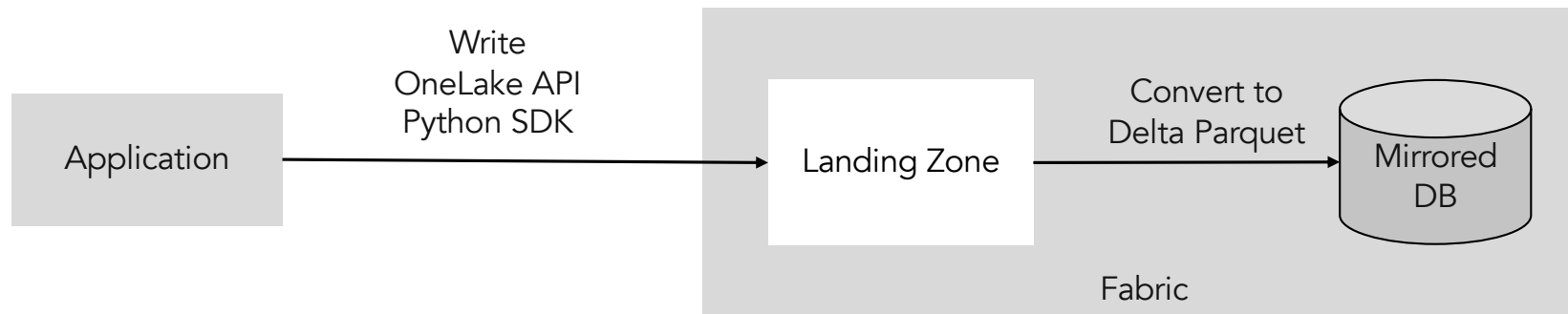


Move Data – Mirroring

- Types of Mirroring:
 - Database mirroring – Replicate database tables.
 - Azure SQL Database, PostgreSQL Flexible Server, SQL MI, On-premise SQL Server, Fabric SQL Database, Databricks.
 - Cosmos DB, Google BigQuery, Snowflake, Databricks.
 - Metadata mirroring – Moves schema, table names but not data.
 - Open mirroring – Based on Delta Lake table format.
- Items created by mirroring
 - Mirrored database.
 - SQL analytics endpoint.

Open Mirroring

- Applications can write data changes directly to mirrored database.



HANDS-ON : Mirroring

- Mirror Azure PostgreSQL database to Fabric.
- Make changes to the source database and check the modified data in Fabric using SQL analytics endpoint.
- Prerequisites:
 - Enable System-assigned Managed Identity (SAMI).
 - WAL_LEVEL should be set to LOGICAL in Server Parameters.
 - azure_cdc extension should be allowed and installed.

Comparison

Capability	 Mirroring	 Copy job	 Copy Activity (Pipeline)	 Eventstream
Sources	Databases + 3 rd party integration into Open Mirroring	All supported data sources and formats	All supported data sources and formats	25+ sources and all formats
Destinations	Tabular format in Fabric OneLake (readonly)	All supported destinations and formats	All supported destinations and formats	4+ destinations
Flexibility	Turn-key setup with fixed behavior	Simple to use + Advanced knobs	Advanced and fully customized knobs	Simple and customizable options
Custom scheduling				
Table and Column management				
Copy behavior: Append, Upsert, Override				
Advanced observability + auditing				
Copy modes				
CDG-based continuous replication				
Batch or bulk copy				
Native support for Incremental copy (watermark-based)				
Copy using user defined query				
Use cases				
Continuous Replication for analytics and reporting				
Metadata driven ELT/ETL for data warehousing				
Data consolidation				
Data migration / Data backup / Data sharing				

Fabric Dataflow

www.3aayaam.in

Transform Data

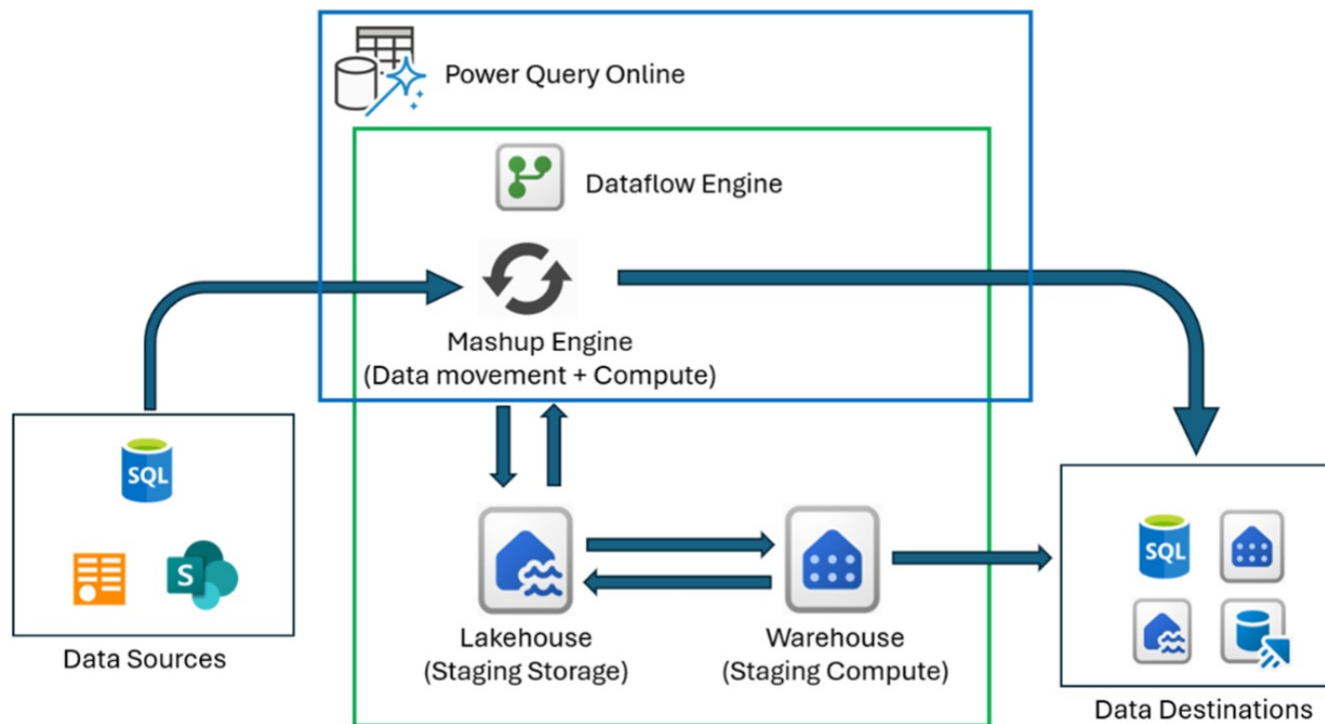
- Data Transformation:
 - Dataflow activity – Visual transformation.
 - Notebook activity – Spark notebooks.
 - Spark Job Definition activity – Spark code.
 - SQL Script activity – DML & DDL. Fabric SQL, Fabric DWH, Azure SQL DB, Azure SQL Managed Instance (MI), Oracle, SQL Server, Azure Synapse Analytics, Snowflake.

Transform Data – Dataflow Gen2

- Data transformation without writing code.
- No Code ETL experience.
- Ingest, transform & load data without writing any code.
- Dataflow Gen 2 is newer version of Power BI Dataflow (Dataflow Gen 1).
- Can be used inside of a pipeline.
- Following are the supported destinations:
 - Azure SQL Databases.
 - Data Explorer.
 - ADLS Gen2.
 - Fabric – Lakehouse tables, Warehouse, KQL database, SQL database.

Dataflow Architecture

- Uses Power Query Mashup Engine.

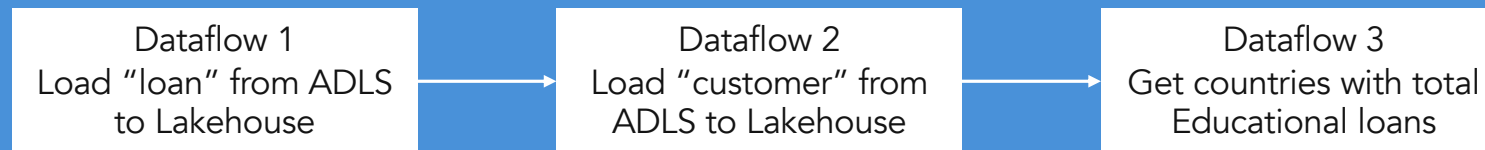


Dataflow - Refresh

- Full refresh – Runs the transformation from the beginning.
 - Reads ALL data from sources.
 - Applies defined transformations on all data.
 - Overwrites or appends data to target.
- Incremental refresh – Runs transformation only on the changed data.
 - Needs datetime column to capture incremental changes.
 - Supported destinations – Fabric Lakehouse, Fabric Warehouse and Azure SQL database.
- Let's explore Dataflow Gen2 UI.

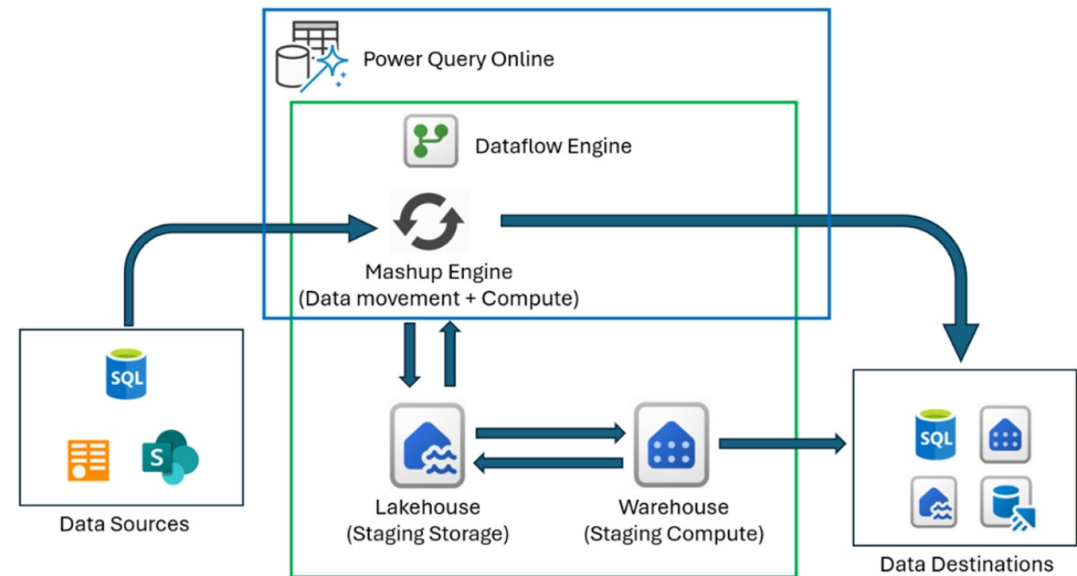
HANDS-ON : Dataflow Gen2

- Dataflow 1 - Read from ADLS (loan & customer) and load to Lakehouse.
- Dataflow 2 – Find the country that has taken highest education loan.
- Run both the flows one after the another in a Pipeline.



Dataflow – Performance & Scale

- Move : Fast Copy. Engine type is “CopyActivity”.
- Process : Query Folding (predicate pushdown).
 - Push as much query evaluation as possible to data source.
 - Process the result set using Power Query. Reduces data movement.
 - Full, Partial or None.
- Process : Staging data and compute.
 - Compute of staging can be used to break large datasets and complex transformations into smaller steps. It reduces overall processing time.



EXERCISE : Dataflow Gen2

- Load all the following files in their corresponding Lakehouse tables. All table names should have _lh at the end.
- Run the following use cases. Output of all should be saved in their corresponding Lakehouse tables. Run all use cases one after the other in a pipeline.
 - List all cross-country transactions.
 - Find top 5 cross-country transactions by amount. (Sum of all transactions from source country to destination country).
 - Which are the top 10 countries that invest in stock market trading (ledger_type = 'Equity') by volume. What is the corresponding invested amount for each countries.
 - Find total amount for each loan type for individual countries.

Comparison

	Pipeline copy activity	Copy job	Dataflow Gen 2	Eventstream	Spark
Use case	Data lake and data warehouse migration, data ingestion, lightweight transformation	Data Ingestion, Incremental copy, Replication, Data Lake and Data Warehouse migration, lightweight transformation	Data ingestion, data transformation, data wrangling, data profiling	event data ingestion, event data transformation	Data ingestion, data transformation, data processing, data profiling
Transformation complexity	Low: lightweight - type conversion, column mapping, merge/split files, flatten hierarchy	Low: lightweight - type conversion, column mapping, merge/split files, flatten hierarchy	Low to high: 300+ transformation functions	Low: lightweight	Low to high: support for native Spark and open-source libraries

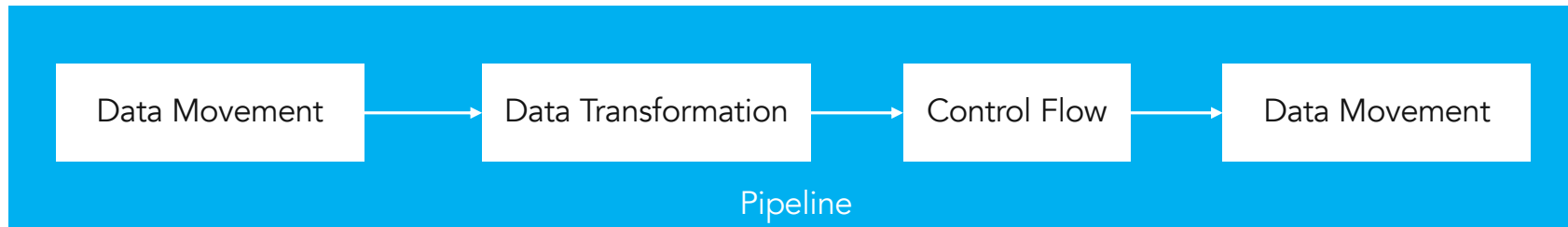
Comparison

	Pipeline copy activity	Copy job	Dataflow Gen 2	Eventstream	Spark
Data volume	Low to high	Low to high	Low to high	Medium to High	Low to high

Fabric Pipeline

www.3aayaam.in

Orchestration - Pipeline

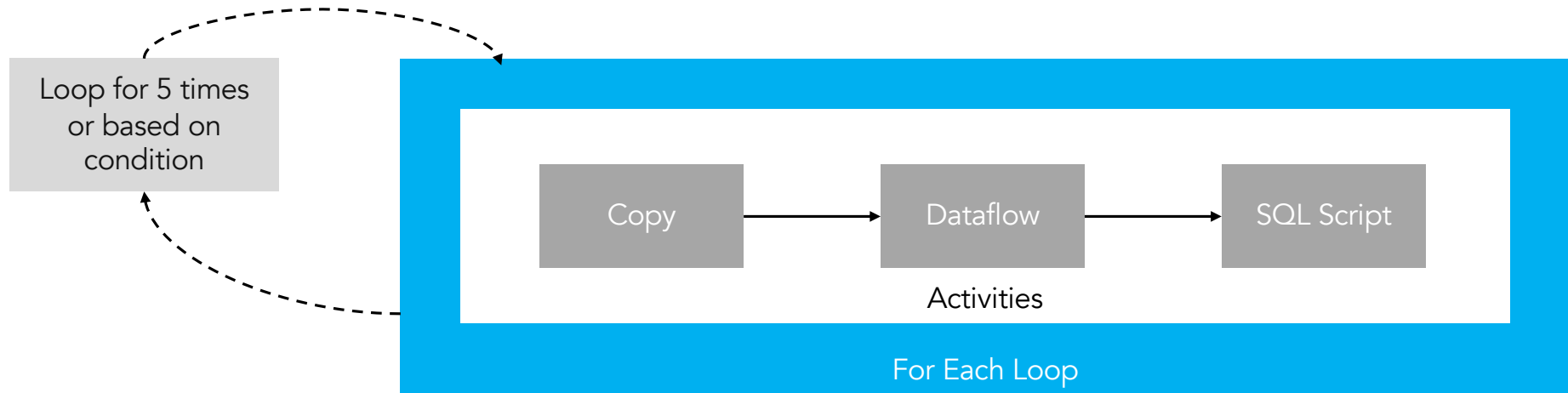


- No-Code data orchestration.
- Pipeline activities:
 - Data movement – Copy Job, Copy Data Activity.
 - Data transformation - Copy Data Activity, Dataflow Gen2, Fabric Spark Notebook, Spark Job Definition, Stored Procedure, SQL Scripts.

Orchestration - Pipeline

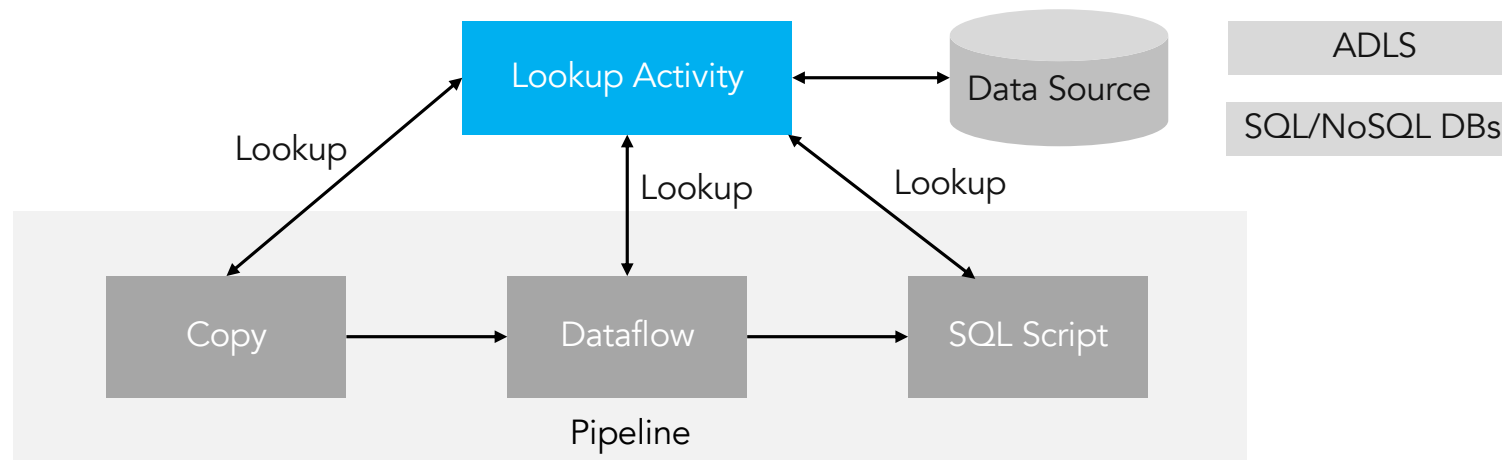
- Pipeline activities - Control Flow
 - ForEach – Iterate one or more activities multiple no. of times.
 - If condition – If logic.
 - Filter – Used for filter expression.
 - Lookup activity – Reads from available connections. It can read and return the content of a configuration file or table. It can also return the result of a query or stored procedure execution.
 - KQL activity – Allows to run query in Kusto Query Language (KQL).
 - Fail – Used to show error message and error code.
 - Set variable – Set the value of an existing variable.
 - Until – Similar to while loop
 - Wait – Wait for certain time.
 - Switch – Similar to switch + case.

Orchestration - ForEach



- All activities inside the ForEach loop will execute multiple times based on the condition.
- Activities inside of loop can run in parallel or one after the other (sequential).
- Batch Count – Can be configured to choose the no. of input items that can be processed together in the loop.

Orchestration - Lookup



- Lookup supported data sources.
- Can return a max. of 5000 rows, output size of 4 MB.

Orchestration - Pipeline

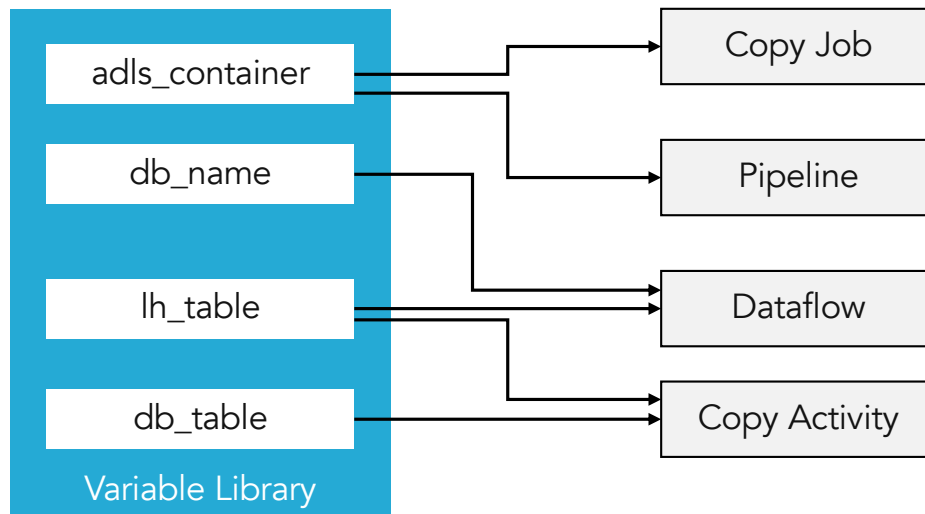
- Pipeline activities - Control Flow
 - Databricks activity – Notebook, PySpark, Spark Job can be run.
 - Azure Batch activity – To run command in Azure Batch instance.
 - Web activity – Can be used to make Web API (REST) call from ADF or Synapse.
 - Azure Function activity – Can be used to run functions.

Pipeline Parameters Expressions

- Parameterization of data pipeline. Templatize a pipeline.
- Parameters have to be set before the start of pipeline.
- Reference Pipeline parameter :
 - `@pipeline().parameters.<variable-name>`
- Reference Pipeline Variable
 - `@variables('<var-name>')`
- Reference previous Activity output :
 - `@activity('<previous-activity-name>').output.<value-name>`

Variable Library

- Define variables at Workspace level.



HANDS-ON : Pipeline

- Create Pipeline to read multiple ADLS folders and load them to multiple Lakehouse tables. Lakehouse table names should be the name of the folder followed by underscore "lh".

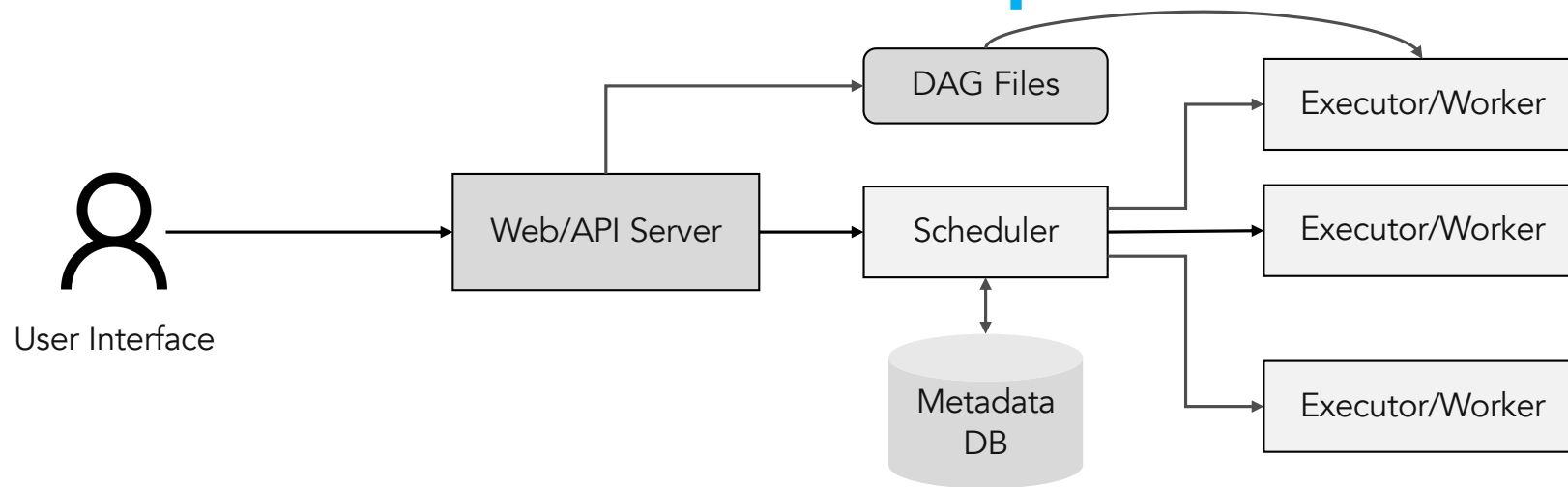
EXERCISE : Pipeline

- Create a configuration table in Azure PostgreSQL with the following columns and values.

Data Source	Source Type	Destination Type
customer	adls	db table
account	adls	db table
loan	adls	db table
transaction	adls	lakehouse
ledger	adls	lakehouse

- Read all rows from the table and load the tables as per source and destination type.
- After all tables are loaded perform the following transformation using Dataflow in the pipeline and load to a Lakehouse destination.
 - Total account balance, loan amount, total amount in ledger for "Expense" type and cross-country transaction for each customer.

Orchestration – Apache Airflow



- Apache Airflow job for orchestration.
- Code based data orchestration – Python DAGs.
- Git integration.

Apache Airflow Server/Compute

- Starter Pool – 4 vCPU & 16 GB RAM per node.
- Custom Pool – Small (2 vCPU + 8GB RAM) or Large (4 vCPU + 16GB RAM) nodes.
 - Starts with 3 nodes – 1 web, 1 scheduler, 1 worker.
 - Autoscaling – Scale up and down worker nodes when needed.
 - Small Pool – Min 5 CUs. Large Pool – 10 CUs.

Pool type	Intended use	Idle behavior	Idle cost	Job run delay
Starter pool	Dev/test or infrequent jobs	Auto-shutdown after ~20 min	No cost when idle	Cold start (~3–5 min) after idle
Custom pool	Production, 24/7 scheduling	Always on (unless paused)	Constant base cost	No delay

Pipeline CI/CD

- Pipeline activities - Control Flow

Summary

www.3aayaam.in

- Fabric Data Factory introduction
- Copy Job to move data
- Copy Activity to move data
- Mirroring to move data
- Dataflow Gen2 for transformation
- Pipeline for orchestration
- Pipeline Parameters
- Apache Airflow